

Arrays

Composite data types group multiple basic types together into a single, organized structure. The most fundamental composite type in Go is the **Array**.

1. The Strict Definition of a Go Array

In languages like **JavaScript**, an "array" or "list" can dynamically grow, shrink, and mix different data types. In Go, arrays are extremely strict:

- They hold a sequence of elements that are all the **exact same type** (e.g., all integers, or all strings).
- They have a **fixed length**. You must decide the exact size of the array before the code even compiles, and it can never change while the program is running.
- They use **Zero-Based Indexing**, meaning the first item is always at position **0**.

2. Declaration and Zero-Value Safety

When you declare an array in Go, you must state the size and the type inside square brackets.

```
var arr [5]int
```

- Go is designed for memory safety. **It does not leave this array empty** or full of junk data. It initializes all 5 slots to the type's **"Zero Value"**. In the case of **int**, the array becomes **[0, 0, 0, 0, 0]**.

3. Array Literals and the ... Shortcut

If you already know exactly which data needs to go into the array right away, you can use an **Array Literal** to declare and fill it in a single step.

```
x := [5]int{10, 20, 30, 40, 50}
```

If you are typing out a long list of default values and don't want to manually count them all to put a number in the brackets, Go gives you a shortcut: the **Ellipsis (...)**.

```
x := [...]int{10, 20, 30, 40, 50, 60, 70, 80, 90, 100}
```

- The ... tells the Go compiler **to count the items inside the curly brackets for you** and lock the array size to that exact number (which would be **10** in this case).

4. Iterating Through Arrays

The standard way to loop through an array is using Go's **for...range loop** (already discussed in loops).